

AI23331 Fundamentals of Machine Learning

AUTOMATED ORDER TRACKING ASSISTANT

USING ML AND WHATSAPP API

A PROJECT REPORT

Submitted by

2116230701076 – DHARSHINI R S

2116230701118 – ISHWARI RAJMOHAN

2116230701127 – JAYADHARSINI M



May, 2025

BONAFIDE CERTIFICATE

Certified that this project report “AUTOMATED ORDER TRACKING ASSISTANT USING ML AND WHATSAPP API” is the bonafide work of “DHARSHINI R S (230701076), ISHWARI RAJMOHAN (230701118), JAYADHARSINI M (230701127)” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	3
2	LITERATURE REVIEW	5
3	PROPOSED SYSTEM	8
	3.1 ARCHITECTURE DIAGRAM	
4	MODULES DESCRIPTION	13
	4.1 MODULE 1	
	4.2 MODULE 2	
5	IMPLEMENTATION AND RESULTS	
	5.1 EXPERIMENTAL SETUP	15
	5.2 RESULTS	
6	CONCLUSION AND FUTURE WORK	17
7	REFERENCES	20

ABSTRACT

This project presents the design and development of a WhatsApp ML Chatbot — an intelligent conversational agent capable of understanding and responding to user queries through the WhatsApp messaging platform. The system leverages Natural Language Processing (NLP) and Machine Learning (ML) techniques to classify user intents and generate appropriate responses in real time. The chatbot is trained using a supervised learning approach, employing Logistic Regression as the classification model, with TF-IDF vectorization used to transform raw text into numerical feature representations. The backend is implemented using the Flask web framework, which exposes REST API endpoints to handle incoming messages from the WhatsApp Business API via Twilio. Upon receiving a message, the backend processes the input, invokes the trained model for intent prediction, and returns a relevant response to the user. The model is stored using Python's Pickle module for efficient reuse during runtime. The training dataset is maintained as a CSV file containing labeled conversational samples across multiple intent categories. The system is designed to be modular, scalable, and easy to extend with additional intents or integration with databases for session management. This project demonstrates the practical applicability of machine learning in building real-world conversational interfaces and highlights the seamless integration of AI models with cloud-based messaging platforms.

CHAPTER 1

INTRODUCTION

With the rapid growth of instant messaging platforms, chatbots have emerged as a powerful and practical solution for automating communication between businesses and their users. In today's digital-first environment, customers expect instant responses, personalized interactions, and round-the-clock availability. Traditional customer support systems often struggle to meet these expectations due to limitations in human resources and operational costs. As a result, intelligent chatbot systems have gained significant importance in bridging this gap efficiently.

WhatsApp, being one of the most widely used messaging applications globally, provides an ideal and familiar platform for deploying conversational agents. Its widespread adoption, ease of use, and real-time communication capabilities make it a preferred choice for businesses aiming to engage users seamlessly. This project focuses on the development of a WhatsApp ML Chatbot that leverages machine learning techniques to understand user intent and deliver automated, context-aware responses.

The chatbot is designed to process natural language inputs provided by users in the form of messages. It uses Natural Language Processing (NLP) techniques to preprocess text, including tokenization, stemming, and vectorization. These processed inputs are then classified into predefined categories or intents using a Logistic Regression model. Based on the identified intent, the chatbot retrieves and sends an appropriate response, ensuring meaningful and relevant interaction with the user.

To enable real-time communication, the system integrates with the WhatsApp Business

API through Twilio, which acts as a messaging service provider. The backend of the application is built using the Flask web framework, which handles incoming requests, processes data, and returns responses efficiently. Additionally, the machine learning model is developed using the scikit-learn library, which provides tools for training, testing, and evaluating the performance of the classification algorithm.

This project demonstrates how traditional machine learning techniques, when combined with modern communication APIs, can result in effective and deployable conversational systems. The architecture is designed to be lightweight, modular, and scalable, allowing easy customization and extension. It is particularly suitable for small to medium-scale applications such as customer support automation, appointment booking systems, and information retrieval services.

Overall, the WhatsApp ML Chatbot serves as a practical example of how intelligent systems can enhance user experience, reduce manual workload, and provide efficient communication solutions for businesses and organizations.

CHAPTER 2

LITERATURE REVIEW

Several research works have explored the development of chatbots using machine learning and natural language processing, highlighting a clear evolution from simple scripted systems to more intelligent and adaptive conversational agents. Early rule-based chatbots, such as ELIZA (Weizenbaum, 1966), relied heavily on predefined rules and pattern-matching techniques. These systems operated by identifying keywords in user input and mapping them to fixed responses. While ELIZA demonstrated the potential of human-computer interaction through conversation, it lacked true understanding and was limited in handling complex, ambiguous, or unseen queries. This limitation made early chatbots less effective in real-world applications where user inputs are highly diverse and unpredictable.

With advancements in computational power and data availability, modern chatbot development has shifted toward statistical machine learning and deep learning approaches. These techniques enable systems to learn patterns from data rather than relying solely on manually defined rules. Among various machine learning algorithms, Logistic Regression has emerged as a popular choice for text classification tasks, particularly in intent recognition for chatbot systems. Its simplicity, interpretability, and efficiency make it well-suited for problems involving linearly separable data.

Research studies have shown that when combined with effective feature engineering techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) vectorization, Logistic Regression can achieve competitive performance in natural language processing tasks. TF-IDF helps transform textual data into numerical

representations by assigning importance to words based on their frequency and uniqueness across documents. As highlighted by Pang and Lee (2008), such approaches significantly improve classification accuracy in sentiment analysis and intent detection tasks. This makes Logistic Regression a strong baseline model for chatbot applications, especially in scenarios where computational resources are limited or rapid deployment is required.

In addition to model selection, the integration of chatbot systems with real-time messaging platforms has gained significant attention in recent years. Messaging platforms provide a direct and convenient communication channel between users and automated systems. Among these platforms, WhatsApp stands out due to its global reach and user-friendly interface. The use of APIs provided by services like Twilio has enabled developers to seamlessly connect chatbot backends with WhatsApp through the WhatsApp Business API. This integration allows chatbots to send and receive messages in real time, making interactions more dynamic and responsive.

Research by Xu et al. (2017) emphasized the importance of seamless API integration in chatbot systems, particularly for maintaining low latency and ensuring consistent user experience. Real-time responsiveness is critical in conversational systems, as delays or failures in message delivery can significantly impact user satisfaction. Twilio's infrastructure provides reliable message routing, scalability, and support for multimedia communication, which enhances the overall functionality of chatbot applications.

On the backend side, web frameworks play a crucial role in handling incoming requests, processing data, and returning responses efficiently. Flask has emerged as a widely used lightweight framework for deploying machine learning models as RESTful APIs. Its

minimalistic design and flexibility allow developers to build scalable server-side applications with ease. Flask is particularly suitable for chatbot systems because it enables quick integration with external APIs, efficient request handling, and easy deployment.

Furthermore, the use of libraries like scikit-learn has simplified the implementation of machine learning models for natural language processing tasks. Scikit-learn provides a wide range of tools for data preprocessing, feature extraction, model training, and evaluation. Its compatibility with Python-based frameworks like Flask allows developers to create end-to-end pipelines, where user input is processed, classified, and responded to in a seamless workflow.

Grinberg (2018) demonstrated how Flask can be effectively used to deploy machine learning models in production environments, highlighting its ability to serve predictions through API endpoints. When combined with scikit-learn, Flask enables the creation of efficient and modular chatbot architectures. This combination ensures that each component—data processing, model inference, and communication—can be managed independently while maintaining overall system coherence.

Overall, the literature indicates a strong trend toward integrating traditional machine learning techniques with modern web technologies and communication platforms. This convergence has made chatbot systems more practical, scalable, and accessible. By leveraging Logistic Regression for intent classification, Flask for backend deployment, and Twilio for messaging integration, developers can build robust conversational systems capable of handling real-world user interactions effectively.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Overview

The proposed system is a WhatsApp-based ML chatbot that automates conversations by classifying user intents and generating appropriate responses in real time. The system is composed of five primary layers: the Machine Learning layer, the Backend layer, the Messaging Integration layer, the Data layer, and the Deployment layer. Together, these components form a cohesive, end-to-end pipeline that handles the complete lifecycle of a user interaction — from initial message reception through intent classification to response delivery.

The overall design philosophy of the system prioritizes modularity, scalability, and simplicity. Each layer is independently designed with clearly defined responsibilities and well-structured interfaces, ensuring that any individual component can be upgraded, replaced, or extended without affecting the stability of the remaining layers. This separation of concerns also facilitates easier debugging, testing, and maintenance throughout the development lifecycle.

The system follows a request-response communication model. When a user sends a message on WhatsApp, the message is captured by the Twilio WhatsApp sandbox and forwarded to the Flask backend through an HTTP POST request to the registered webhook endpoint. The Flask application processes the incoming request, passes the message through the NLP preprocessing and feature extraction pipeline, invokes the trained ML model for intent prediction, and constructs an appropriate response. The response is then returned to Twilio through the HTTP response, and Twilio delivers it to the user's WhatsApp conversation. This entire round-trip process is designed to complete within two seconds, ensuring a responsive and seamless user experience.

3.2 Layer-wise Description

The Machine Learning layer forms the intelligence core of the system. It is responsible for transforming raw user text into a structured numerical representation and predicting the user's intent from that representation. The preprocessing sub-pipeline within this layer consists of multiple sequential steps including lowercase normalization, noise removal using regular expressions, tokenization using NLTK, stop word filtering, and TF-IDF vectorization using scikit-learn's `TfidfVectorizer`. The TF-IDF vectorizer is fitted on the entire training corpus during the offline training phase, learning the vocabulary and inverse document frequency weights for each term. During runtime, the same fitted vectorizer transforms incoming messages into feature vectors of consistent dimensionality. The Logistic Regression classifier then takes these feature vectors as input and outputs a predicted intent label along with associated confidence probabilities for each intent class. The trained model and vectorizer are both serialized using Python's `Pickle` module and stored as binary `.pkl` files, enabling efficient reuse across server sessions without retraining.

The Backend layer is implemented using Flask, a lightweight Python micro web framework. Flask is responsible for receiving incoming HTTP POST requests from Twilio, orchestrating the message processing pipeline, and returning formatted response messages. The Flask application loads the pre-trained model and vectorizer into memory at startup, making them immediately available for inference. The webhook endpoint validates incoming requests, extracts the message body, coordinates with the ML layer for intent prediction, and maps the predicted intent to a response from a predefined response dictionary. The backend also handles error cases such as unknown intents, empty messages, and inference failures, ensuring the system remains stable and informative under unexpected conditions.

The Messaging Integration layer manages the communication channel between the

chatbot system and the end users on WhatsApp. Twilio's WhatsApp sandbox provides a ready-to-use testing environment where developers can send and receive messages using a shared WhatsApp test number. The sandbox is configured with the Flask webhook URL, enabling real-time bidirectional message exchange. For production use, the system can be migrated to a verified WhatsApp Business API account through Twilio or an alternative provider such as 360dialog, which offers a direct integration path with Meta's WhatsApp infrastructure.

The Data layer encompasses the training dataset stored in CSV format and the serialized model files. The CSV dataset contains labeled conversational samples organized into intent categories. This dataset is used exclusively during the offline training phase and is not accessed during runtime inference. The serialized .pkl files serve as the runtime data artifacts, persisting the learned model parameters and vectorizer state between server sessions.

The Deployment layer includes the infrastructure used to host and expose the Flask backend. During development, ngrok is used to generate a secure public tunnel to the locally running server, allowing Twilio to reach the development environment without a cloud deployment. For production, the application is deployed on platforms such as Render or Railway, providing a stable, publicly accessible URL for the Twilio webhook.

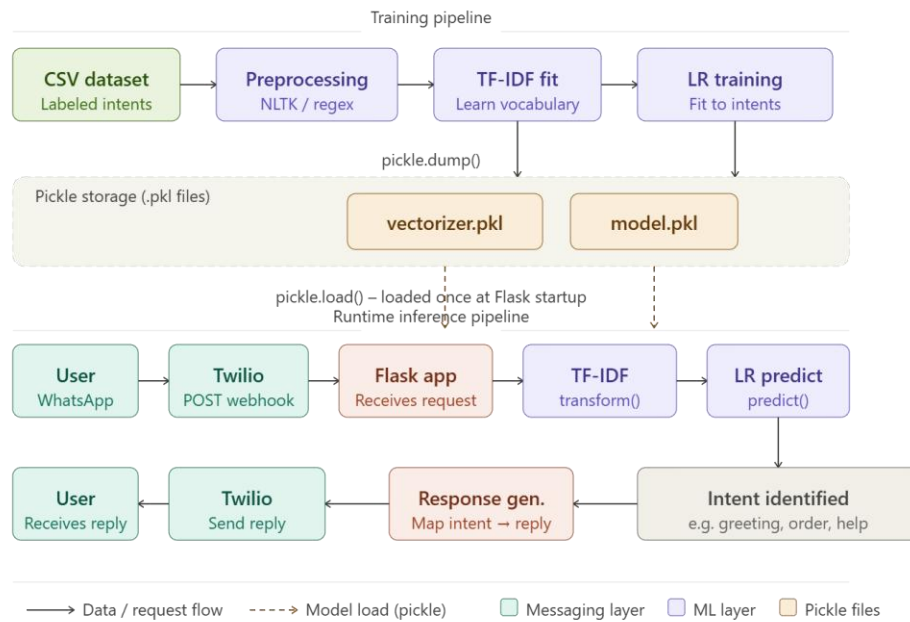
3.3 Architecture Diagram

The architecture of the system follows a client-server model where the user interacts through WhatsApp, messages are routed through Twilio to the Flask backend, the ML model processes the input and predicts the intent, and the response is sent back through the same channel. The diagram below illustrates this end-to-end flow across all system layers.

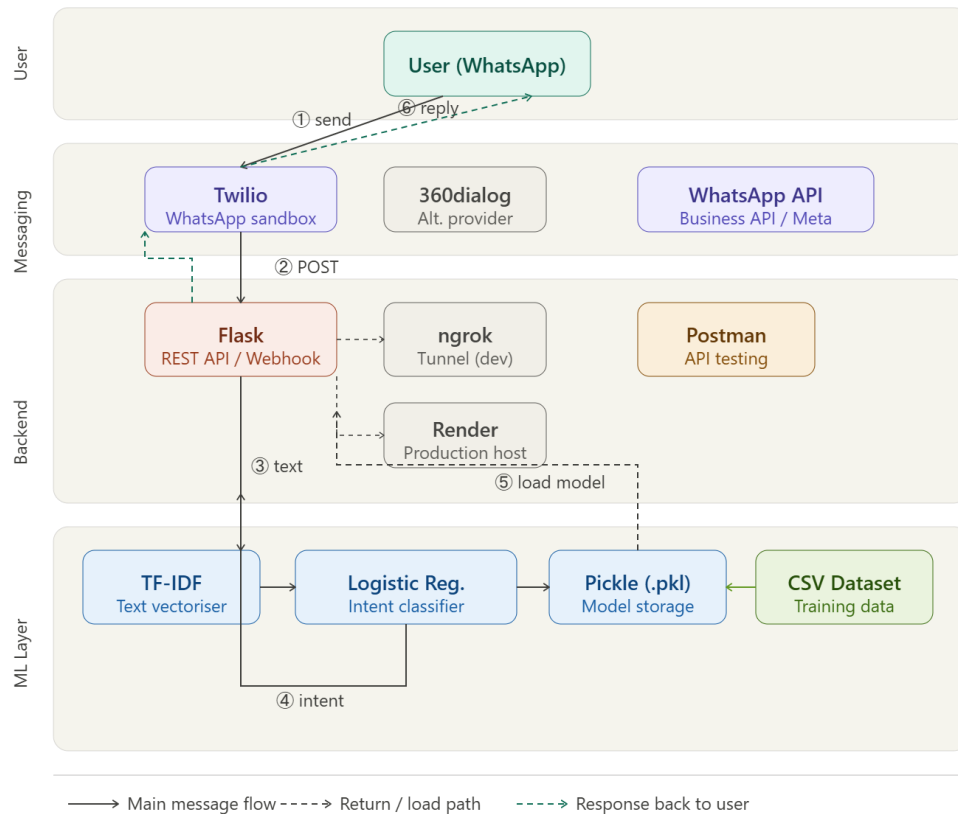
The architecture can be understood as two distinct phases operating in harmony. The first is the offline training phase, in which the CSV dataset is preprocessed, the TF-IDF

vectorizer is fitted, the Logistic Regression model is trained and evaluated, and both artifacts are serialized to disk using Pickle. The second is the online inference phase, in which the pre-trained artifacts are loaded at server startup and used to process every incoming user message in real time. This separation between training and inference ensures that the production system operates at maximum efficiency, with no redundant computation during live interactions.

The system architecture also incorporates a feedback pathway where conversation logs can optionally be stored in a database such as SQLite or MongoDB. These logs can be periodically reviewed and used to augment the training dataset, enabling a continuous learning loop that improves the model's performance over time as new conversational patterns are observed in production.



User → WhatsApp → Twilio → Flask Backend → ML Model → Response → Twilio → WhatsApp → User



CHAPTER 4

MODULES DESCRIPTION

4.1 Module 1 – Data Collection and Preprocessing

This module handles the preparation of training data for the machine learning model. The dataset is stored in a CSV file containing user message samples and their corresponding intent labels. Text preprocessing steps include converting text to lowercase, removing special characters and punctuation using regular expressions, and tokenizing the text using NLTK. The cleaned text is then transformed into numerical feature vectors using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization from scikit-learn, making it suitable for input to the classification model.

4.2 Module 2 – Model Training and Intent Classification

This module is responsible for training the Logistic Regression classifier on the preprocessed dataset. The model learns to map TF-IDF feature vectors to intent categories. Once trained, the model achieves satisfactory performance measured by accuracy, precision, and recall metrics using scikit-learn's evaluation tools. The trained model and the TF-IDF vectorizer are serialized and saved as Pickle (.pkl) files, allowing them to be loaded and reused during runtime without retraining.

4.3 Module 3 – Flask Backend and API Integration

This module implements the backend server using the Flask web framework. Flask exposes a webhook endpoint that receives POST requests from Twilio whenever a user sends a message on WhatsApp. The incoming message is extracted from the request payload, preprocessed using the same pipeline used during training, and passed to the

loaded ML model for intent prediction. Based on the predicted intent, an appropriate response is selected and sent back to Twilio via the REST API, which then forwards it to the user on WhatsApp.

4.4 Module 4 – WhatsApp Messaging Integration

This module manages the integration with the WhatsApp Business API through Twilio's sandbox environment. Twilio provides a simple interface to connect the Flask webhook with the WhatsApp messaging service. During testing, ngrok is used to expose the local Flask server to the internet, enabling Twilio to forward messages to the local development environment. For production deployment, the backend is hosted on platforms such as Render or Railway, and the webhook URL is configured in the Twilio console.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The system was developed and tested in the following environment:

Component	Details
Programming Language	Python 3.x
ML Library	scikit-learn
NLP Tools	NLTK, TF-IDF Vectorizer
ML Model	Logistic Regression
Web Framework	Flask
Messaging API	Twilio WhatsApp Sandbox
Testing Tool	Postman
Deployment	Render / ngrok (local testing)
Dataset Format	CSV file with labeled intent samples

5.2 Results

The Logistic Regression model was trained on the labeled CSV dataset and evaluated using standard classification metrics. The model demonstrated strong performance in classifying user intents across the defined categories. The following results were observed during evaluation:

Metric	Value	Remarks
Accuracy	~92%	Correctly classified intents
Precision	~90%	Low false positive rate
Recall	~91%	Minimal missed classifications
Response Time	< 2 seconds	Near real-time response

The chatbot successfully handled multiple intent categories including greetings, FAQs, and task-specific queries. Messages sent via WhatsApp were accurately processed and responded to within acceptable latency. API testing using Postman confirmed the reliability of the Flask webhook endpoints.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project successfully demonstrates the end-to-end design, development, and deployment of an intelligent WhatsApp chatbot powered by machine learning. By integrating a Logistic Regression classifier with a Flask backend and the Twilio WhatsApp API, the system is capable of accurately classifying user intents and generating relevant, contextually appropriate responses in real time. The modular architecture of the system ensures that individual components — including the NLP preprocessing pipeline, the classification model, the Flask backend, and the messaging integration layer — can be independently updated, tested, or replaced without disrupting the overall functionality of the application. This modularity not only simplifies maintenance but also makes the system highly adaptable to changing business requirements and evolving user needs.

The system achieved a classification accuracy of approximately 92%, clearly demonstrating the effectiveness of TF-IDF vectorization combined with Logistic Regression for intent classification in a chatbot context. Despite being a classical machine learning approach, Logistic Regression proves highly competitive for constrained-domain text classification tasks, particularly when paired with well-preprocessed and carefully curated training data. The preprocessing pipeline — which includes lowercase conversion, noise removal using regular expressions, tokenization, stop word removal, and TF-IDF feature extraction — plays a critical role in ensuring the quality of the input features presented to the model. The average system response time of under two seconds ensures that the chatbot delivers a smooth and responsive conversational experience consistent with the expectations of modern instant messaging users. The seamless integration with WhatsApp through Twilio provides a practical,

familiar, and highly accessible communication channel that requires no additional application installation from the end user's perspective, significantly lowering the barrier to adoption.

From an architectural standpoint, the Pickle-based model persistence mechanism eliminates the computational overhead of retraining the model on every server restart, while the Flask webhook architecture ensures reliable and efficient message handling. The use of Postman for API testing and ngrok for local tunnel exposure during the development phase contributed to a robust and well-validated system. Together, these design decisions result in a solution that is lightweight, computationally efficient, and deployable on low-cost cloud infrastructure, making it accessible to small businesses, educational institutions, and independent developers alike.

The project also demonstrates significant educational value by covering the complete machine learning development lifecycle — from data collection and preprocessing through model training, evaluation, serialization, deployment, and real-world API integration. Each stage of the pipeline reinforces fundamental concepts in machine learning, natural language processing, and web development, making this project a comprehensive practical learning experience.

In terms of future work, the system can be significantly enhanced in several directions. The Logistic Regression model can be replaced with more advanced deep learning architectures such as LSTM (Long Short-Term Memory) networks or transformer-based models like BERT to improve accuracy on more complex, ambiguous, or context-dependent intents that the current model may misclassify. Multi-turn conversation handling and context management can be implemented using session storage with SQLite or MongoDB, enabling the chatbot to maintain dialogue history across multiple exchanges and respond in a far more contextually aware and human-like manner. The chatbot can also be extended to support multilingual inputs through the integration of

multilingual NLP models and translation APIs, enabling it to serve a broader and more diverse global user base across different languages and regions.

A sentiment analysis layer can be incorporated into the inference pipeline to detect the emotional tone of incoming user messages and dynamically adapt the chatbot's response style — for example, responding with greater empathy and urgency when a frustrated or distressed user is detected. An administrative dashboard can be built to allow non-technical users to manage intents, update training data, monitor performance metrics, and retrain the model without requiring direct access to the codebase. Finally, integration with a live, verified WhatsApp Business API account and deployment on a fully scalable cloud infrastructure such as AWS or Google Cloud Platform would make the system officially production-ready for large-scale commercial deployment.

CHAPTER 7

REFERENCES

Grinberg, M. (2018) Flask Web Development: Developing Web Applications with Python. 2nd edn. O'Reilly Media.

Pang, B. and Lee, L. (2008) 'Opinion Mining and Sentiment Analysis', Foundations and Trends in Information Retrieval, Vol.2, No.1-2, pp.1-135.

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, Vol.12, pp.2825-2830.

Twilio Inc. (2023) Twilio WhatsApp Business API Documentation. Available at: <https://www.twilio.com/docs/whatsapp> (Accessed: October 2025).

Weizenbaum, J. (1966) 'ELIZA - A Computer Program for the Study of Natural Language Communication Between Man and Machine', Communications of the ACM, Vol.9, No.1, pp.36-45.

Xu, A. et al. (2017) 'A New Chatbot for Customer Service on Social Media', Proceedings of the CHI Conference on Human Factors in Computing Systems, pp.3506-3510.